available partway through the execution of other tasks, so all tasks must be running concurrently.

With the block-based, forward-propagation approach out of the way, let's look at the second approach to computing the partial sums of a range.

Implementing the Incremental Pairwise Algorithm for Partial Sums

This second approach to calculating the partial sums by adding elements increasingly further away works best where your processors can execute the additions in lockstep. In this case, no further synchronization is necessary because all the intermediate results can be propagated directly to the next processor that needs them. But in practice you rarely have such systems to work with except for those cases where a single processor can execute the same instruction across a small number of data elements simultaneously with so-called Single-Instruction/Multiple-Data (SIMD) instructions. Therefore, you must design your code for the general case and explicitly synchronize the threads at each step.

One way to do this is to use a *barrier*—a synchronization mechanism that causes threads to wait until the required number of threads has reached the barrier. Once all the threads have reached the barrier, they're all unblocked and may proceed. The C++11 Thread Library doesn't offer such a facility directly, so you have to design one yourself.

Imagine a roller coaster at the fairground. If there's a reasonable number of people waiting, the fairground staff will ensure that every seat is filled before the roller coaster leaves the platform. A barrier works the same way: you specify up front the number of "seats," and threads have to wait until all the "seats" are filled. Once there are enough waiting threads, they can all proceed; the barrier is reset and starts waiting for the next batch of threads. Often, such a construct is used in a loop, where the same threads come around and wait next time. The idea is to keep the threads in lockstep, so one thread doesn't run away in front of the others and get out of step. For an algorithm such as this one, that would be disastrous, because the runaway thread would potentially modify data that was still being used by other threads or use data that hadn't been correctly updated yet.